

Optimisation Techniques for Engineering Design

Prof. Sanjib Kumar Acharyya

Department of mechanical Engineering

Jadavpur University

Presentation :4

Algorithms for Unconstrained multivariable objective function



Unconstrained multivariable objective functions

The objective function is a function of N variables represented by $x_1, x_2, \dots, x_N$.

Minimise, $f(X) = f(x_1, x_2, \dots, x_N)$

The solution of this function is given by vector $X = [x_1, x_2, \dots, x_N]^T = \begin{bmatrix} x_1 \\ x_2 \\ \cdot \\ \cdot \\ x_N \end{bmatrix}$

The **optimum** solution of this function is given by vector $X^* = \begin{bmatrix} x_1^* \\ x_2^* \\ \cdot \\ \cdot \\ x_N^* \end{bmatrix}$

$$\frac{\partial f}{\partial x_1} = \frac{f(x_1 + \Delta x_1, x_2, \dots, x_N) - f(x_1, x_2, \dots, x_N)}{\Delta x_1}$$

Size of vector X , $|X| = \sqrt{x_1^2 + x_2^2 + \dots + x_N^2}$

Minimum value of $f(X) = f(X^*) = f(x_1^*, x_2^*, \dots, x_N^*)$

The gradient vector at any point x^t is represented by $\nabla f(x^t)$ which is an N dimensional vector given as follows:

$$\nabla f(x^{(t)}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_N} \right)^T \bigg|_{x^{(t)}}$$

The first-order partial derivatives can be calculated numerically



$$\frac{\partial f}{\partial x_1} = \frac{f(x_1 + \Delta x_1, x_2, \dots, x_N) - f(x_1, x_2, \dots, x_N)}{\Delta x_1}$$

In a multivariable function, the gradient of a function is not a scalar quantity; instead it is a vector quantity

$$\nabla f(X) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{bmatrix}$$

$$\frac{\partial f}{\partial x_2} = \frac{f(x_1, x_2 + \Delta x_2, \dots, x_N) - f(x_1, x_2, \dots, x_N)}{\Delta x_2}$$

the second-order derivatives in multivariable functions form a matrix, $\nabla^2 f(x^t)$ (better known as the Hessian matrix) given as :

$$\nabla^2 f(x^{(t)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \frac{\partial^2 f}{\partial x_N \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}_{x^{(t)}}$$



Optimality Criteria :

A point x is a stationary point if $\nabla f(x) = 0$. Furthermore, the point is a minimum, a maximum, or an inflection point if $\nabla^2 f(x)$ is positive-definite, negative-definite, or otherwise.

A matrix $\nabla^2 f(x^t)$ is defined to be positive-definite if for any point y in the search space the quantity $y^T \nabla^2 f(x^t) y \geq 0$.

A matrix $\nabla^2 f(x^t)$ is defined to be negative-definite if for any point y in the search space the quantity $y^T \nabla^2 f(x^t) y \leq 0$.

- There are a number of ways to investigate whether a matrix is positive-definite.
- One common way to do this is to evaluate the eigenvalues of the matrix. If all eigenvalues are positive, the matrix is positive-definite.
- The other way to test the positive-definiteness of a matrix is to calculate the principal determinants of the matrix.
- If all principal determinants are positive, the matrix is positive-definite.
- It is worth mentioning here that the negative-definiteness of a matrix A can be verified by testing the positive definiteness of the matrix $-A$.



Unidirectional Search (One dimensional)

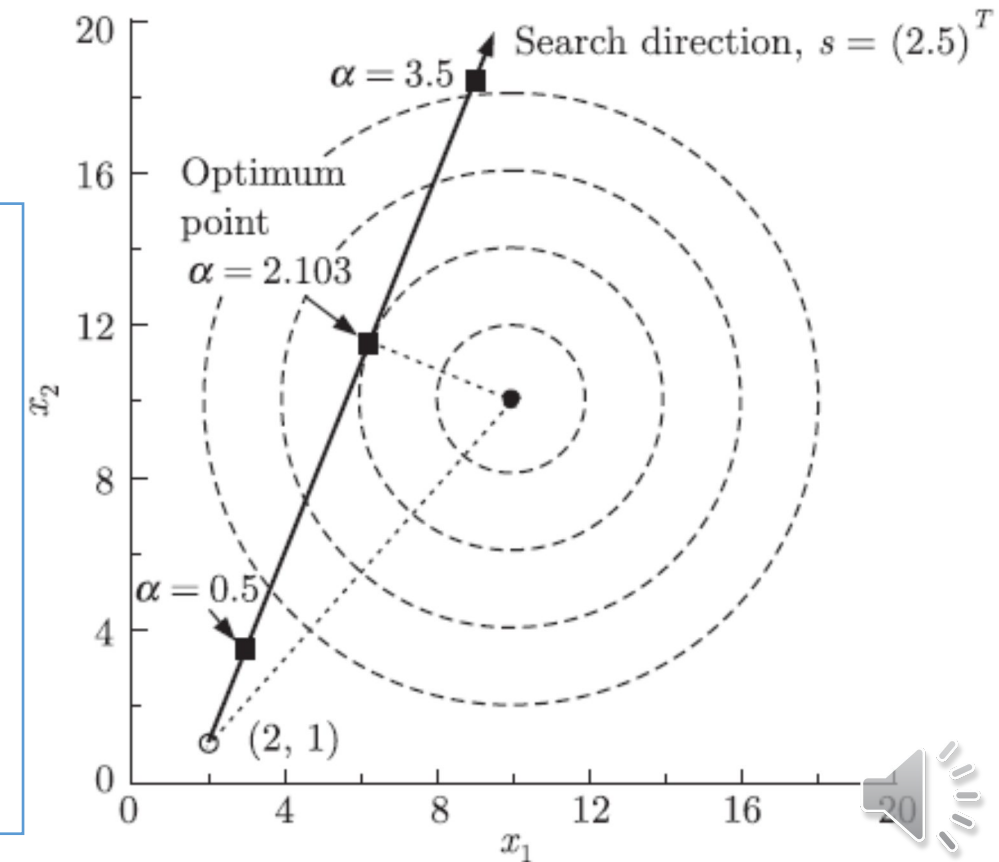
Many multivariable optimization techniques use successive unidirectional search techniques to find the minimum point along a particular search direction.

Usually, a unidirectional search is performed from a point x^t and in a specified direction s^t . That is, only points that lie on a line (in an N -dimensional space) passing through the point x^t and oriented along the search direction s^t are allowed to be considered in the search process.

Any arbitrary point on that line can be expressed as follows:

$$x(\alpha) = x^t + \alpha s^t.$$

- The parameter α is a scalar quantity, specifying a relative measure of distance of the point $x(\alpha)$ from $x(t)$.
- Thus, for a given value of α , the point $x(\alpha)$ can be known. Any positive and negative value of α will create a point on the desired line.
- For each search direction s^t the best point (closest) on that line can be identified by optimizing α using *any single variable optimization algorithm*.
- Such repeated unidirectional search will identify actual minima



Box's Evolutionary Optimization Method : (Direct search)

Box's evolutionary optimization is a simple optimization technique developed by G. E. P. Box in 1957.

The algorithm requires $(2^N + 1)$ points, of which 2^N are corner points of an N -dimensional hypercube centred on the other point.

All $(2^N + 1)$ function values are compared and the best point is identified.

In the next iteration, another hypercube is formed around this best point.

If at any iteration, an improved point is not found, the size of the hypercube is reduced.

This process continues until the hypercube becomes very small.

Algorithm

Step 1 Choose an initial point x^0 and size reduction parameters Δ_i for all design variables, $i = 1, 2, \dots, N$.

Choose a termination parameter ϵ .

Set $x = x^0$. $(x_1^0, x_2^0, \dots, x_N^0)$

Step 2 If $\|\Delta\| < \epsilon$, **Terminate**;

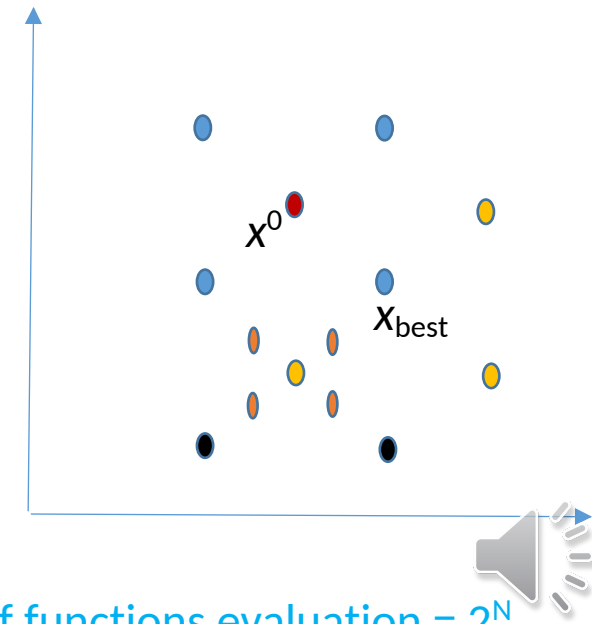
Else create $2N$ points by adding and subtracting $\Delta_i/2$ from each variable at the point x^0 .

Step 3 Compute function values at all $(2^N + 1)$ points. Find the point having the minimum function value. Designate the minimum point to be x_{best} .

Step 4 If $x_{best} = x^0$, reduce size parameters $\Delta_i = \Delta_i/2$ and go to Step 2;

Else set $x^0 = x_{best}$ and go to Step 1., x^0 is always set as the current best point.

Thus, at the end of simulation, x^0 becomes the obtained optimum point.



No. of functions evaluation = 2^N

$$\text{Minimize } f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

the optimum solution will be a set of values of x_1 and x_2 , satisfying the following two equations:

$$x_1^2 + x_2 = 11, \quad x_1 + x_2^2 = 7.$$

$$X_{\text{opt}} = (x_1 = 3, x_2 = 2)$$

Box's Evolutionary Optimization algorithm :

Step 1 We choose an initial point $x^{(0)} = (1, 1)^T$ and a size reduction parameter, $\Delta = (2, 2)^T$. We also choose $\epsilon = 10^{-3}$ and initialize $\bar{x} = x^{(0)} = (1, 1)^T$.

Step 2 Since $\|\Delta\| = 2.828 > 10^{-3}$, we create a two-dimensional hypercube (a square) around $x^{(0)}$:

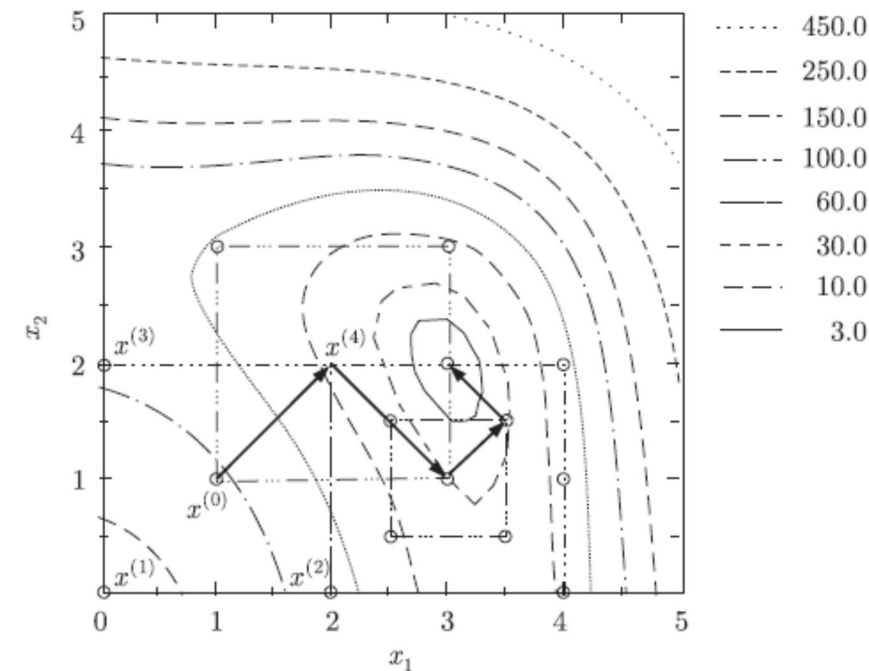
$$x^{(1)} = (0, 0)^T, \quad x^{(2)} = (2, 0)^T, \quad x^{(3)} = (0, 2)^T, \quad x^{(4)} = (2, 2)^T.$$

Figure 3.4 shows this square with five chosen points.

Step 3 The function values at the five points are

$$\begin{aligned} f(x^{(0)}) &= 106, & f(x^{(1)}) &= 170, & f(x^{(2)}) &= 74, \\ f(x^{(3)}) &= 90, & f(x^{(4)}) &= 26. \end{aligned}$$

The minimum of the five function values is 26 and the corresponding point is $x^{(4)} = (2, 2)^T$. Thus we designate $\bar{x} = (2, 2)^T$.



Step 4 Since $\bar{x} \neq x^{(0)}$, we set $x^{(0)} = (2, 2)^T$ and go to Step 2. This completes one iteration of the Box's evolutionary optimization method. Figure 3.4 shows how the initial point $(1, 1)^T$ has moved to the point $(2, 2)^T$ in one iteration.

Step 2 The quantity $\|\Delta\| = 2.828$ is not small and therefore, we create a square around the point $(2, 2)^T$ by adding and subtracting Δ_i to the variable x_i :

$$x^{(1)} = (1, 1)^T, \quad x^{(2)} = (3, 1)^T, \quad x^{(3)} = (1, 3)^T, \quad x^{(4)} = (3, 3)^T.$$

Step 3 The corresponding function values are

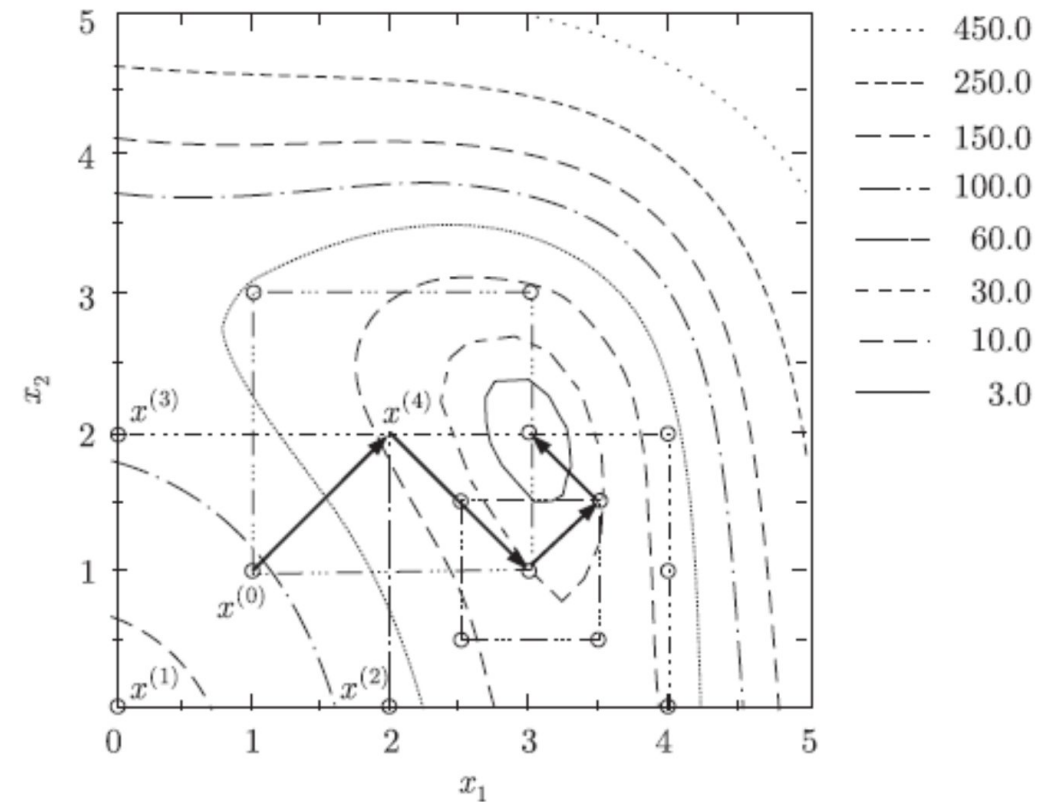
$$\begin{aligned} f(x^{(0)}) &= 26, & f(x^{(1)}) &= 106, & f(x^{(2)}) &= 10, \\ f(x^{(3)}) &= 58, & f(x^{(4)}) &= 26. \end{aligned}$$

The minimum point is $\bar{x} = (3, 1)^T$ having the function value equal to 10.

Step 4 Since $\bar{x} \neq x^{(0)}$, we set the current best point $x^{(0)} = (3, 1)^T$ and proceed to Step 2.

Step 2 The new hypercube around $(3, 1)^T$ is as follows:

$$x^{(1)} = (2, 0)^T, \quad x^{(2)} = (4, 0)^T, \quad x^{(3)} = (2, 2)^T, \quad x^{(4)} = (4, 2)^T.$$



Step 4 Since the new point is the same as the previous best point $x^{(0)}$, we reduce the size parameter $\Delta = (1, 1)^T$ and move to Step 2.

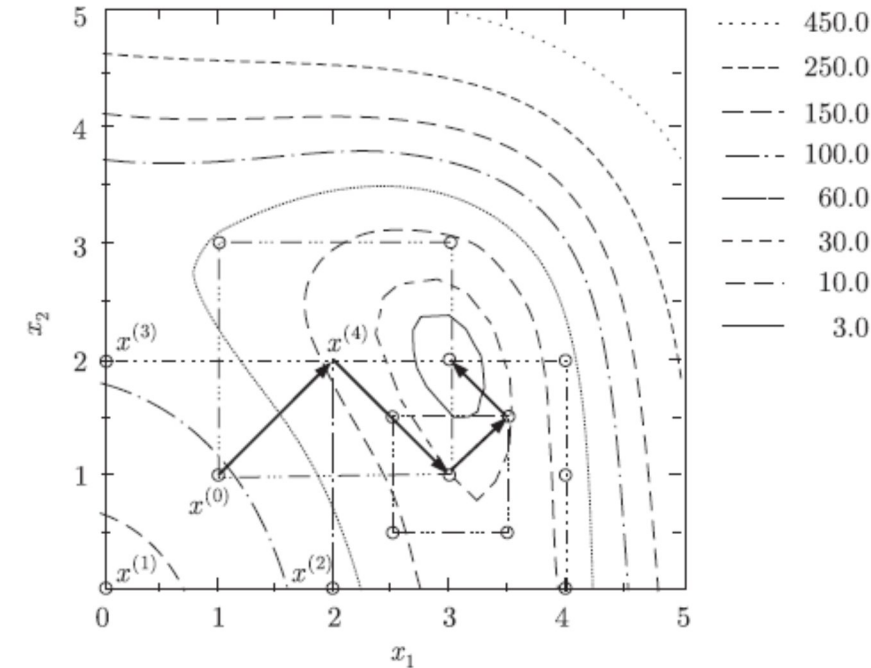
Step 2 The new square with a reduced size around $(3, 1)^T$ is as follows (Figure 3.4):

$$x^{(1)} = (2.5, 0.5)^T, \quad x^{(2)} = (3.5, 0.5)^T,$$

$$x^{(3)} = (2.5, 1.5)^T, \quad x^{(4)} = (3.5, 1.5)^T.$$

Step 3 The minimum of these points is $\bar{x} = x^{(4)} = (3.5, 1.5)^T$ having function value of 9.125.

Step 4 By reducing the size of the rectangle, we get a better point compared to the previous best point. We proceed to Step 2 by setting $x^{(0)} = (3.5, 1.5)^T$.



Convergence criteria : When the value $\|\Delta\|$ becomes smaller than ϵ , the algorithm terminates



Simplex Method :

- In the simplex search method, the number of points in the initial *simplex* is (N+1) for N number of design.
- Selected points should not form a zero-volume N-dimensional hypercube.
- Thus, in a function with two variables, the chosen three points in the simplex should not lie along a line.

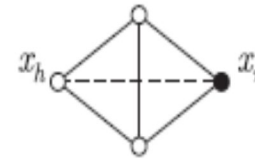
Algorithm :

Step 1 Choose $\gamma > 1$, $\beta \in (0, 1)$, and a termination parameter ϵ . Create an initial simplex of (N+1) points

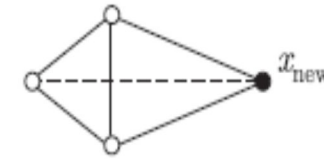
Step 2 Find x_h (the worst point), x_l (the best point), and x_g (next to the worst point).

Calculate

$$x_c = \frac{1}{N} \sum_{i=1, i \neq h}^{N+1} x_i.$$



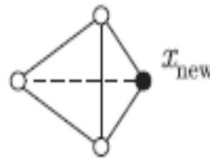
(a) Reflection



$\gamma > 1$
(b) Expansion



$\beta < 0$



$\beta > 0$

(c), (d) Contractions

Step 3 Calculate the reflected point $x_r = 2x_c - x_h$. Set x_l

If $f(x_r) < f(x_l)$, set $x_{\text{new}} = (1 + \gamma)x_c - \gamma x_h$ (expansion);

Else if $f(x_r) \geq f(x_h)$, set $x_{\text{new}} = (1 - \beta)x_c + \beta x_h$ (contraction);

Else if $f(x_g) < f(x_r) < f(x_h)$, set $x_{\text{new}} = (1 + \beta)x_c - \beta x_h$ (contraction).

Calculate $f(x_{\text{new}})$ and replace x_h by x_{new} .

Step 4 If $\left\{ \sum_{i=1}^{N+1} \frac{(f(x_i) - f(x_c))^2}{N + 1} \right\}^{1/2} \leq \epsilon$, **Terminate**;

Else go to Step 2.

- The performance of the above algorithm depends in the values of β and γ .

- Higher value of β and γ yields faster convergence but larger error in optima

Gradient-based Methods :

- Gradient-based methods exploit the derivative information of the function and are usually faster search methods.
- The derivatives can be obtained by numerical computation of objective functions only at two or three neighbouring points.

$$\left. \frac{\partial f(x)}{\partial x_i} \right|_{x^{(t)}} = \left(f(x_i^{(t)} + \Delta x_i^{(t)}) - f(x_i^{(t)} - \Delta x_i^{(t)}) \right) / \left(2\Delta x_i^{(t)} \right), \quad (3.4)$$

$$\left. \frac{\partial^2 f(x)}{\partial^2 x_i^2} \right|_{x^{(t)}} = \left(f(x_i^{(t)} + \Delta x_i^{(t)}) - 2f(x^{(t)}) + f(x_i^{(t)} - \Delta x_i^{(t)}) \right) / \left(\Delta x_i^{(t)} \right)^2, \quad (3.5)$$

$$\begin{aligned} \left. \frac{\partial^2 f(x)}{\partial x_i \partial x_j} \right|_{x^{(t)}} = & \left[f(x_i^{(t)} + \Delta x_i^{(t)}, x_j^{(t)} + \Delta x_j^{(t)}) - f(x_i^{(t)} + \Delta x_i^{(t)}, x_j^{(t)} - \Delta x_j^{(t)}) \right. \\ & - f(x_i^{(t)} - \Delta x_i^{(t)}, x_j^{(t)} + \Delta x_j^{(t)}) \\ & \left. + f(x_i^{(t)} - \Delta x_i^{(t)}, x_j^{(t)} - \Delta x_j^{(t)}) \right] / \left(4\Delta x_i^{(t)} \Delta x_j^{(t)} \right). \end{aligned}$$

$$\nabla f(X) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{bmatrix}$$

$$\nabla^2 f(x^{(t)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \frac{\partial^2 f}{\partial x_N \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}$$

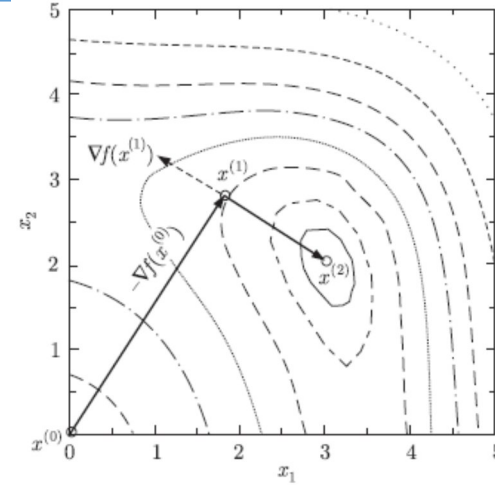


Steps in gradient based methods :

- Most gradient-based methods work by searching along several directions iteratively.
- The algorithms vary according to the way the search directions are defined.
- While at point x^k , best search direction, s^k is identified .
- A unidirectional search is performed along s^k to locate the minimum in this direction.
- This is performed by first writing a representative point along the direction s^k as follows:

$$x^{(k+1)} = x^{(k)} + \alpha^{(k)} \cdot s^{(k)}, \quad | \quad |$$

- where $\alpha^{(k)}$ is the step length. Since $x^{(k)}$ and $s^{(k)}$ are known, the point $x^{(k+1)}$ can be expressed with only one variable $\alpha^{(k)}$.
- Objective function $f(X)$ will be a function of one variable α .
- Thus, a unidirectional search for α_{opt}^k will identify best point along s^k and the new point $x^{(k+1)}$ can be obtained.
- Thereafter, the search is continued from the new point $x^{(k+1)}$ along another search direction $s^{(k+1)}$.



Convergence Criteria : either, $|(x(k+1)) - (x(k))| \leq \epsilon_1$, or, $|f(x(k+1)) - f(x(k))| \leq \epsilon_2$ or, both



Let $x^{(k)}$ be the current point at iteration level k (Initial point for $K=1$)

$$\text{Then, } x^{(k+1)} = x^{(k)} + \alpha^{(k)} \cdot s^{(k)},$$

What is best $s^{(k)}$? **The best possible search direction using function or gradient values at $x^{(k)}$**

1) **Cauchy's Steepest Descent Method** : $s^{(k)} = -\nabla f(x^{(k)})$: $x(k+1) = x(k) - \alpha^{(k)} \cdot \nabla f$,

This technique is effective when x^k is far away from x_{opt} .

2) **Newton's Method** : $s^{(k)} = \frac{-\nabla f(x^k)}{\nabla^2 f(x^k)}$; $x(k+1) = x(k) - \alpha^{(k)}$.

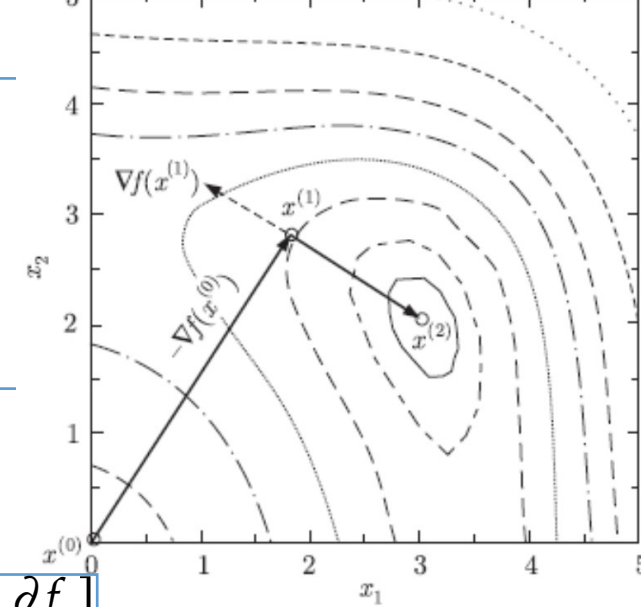
This technique is effective when x^k is very close to x_{opt} .

Marquardt's Method : $s^{(k)} = \frac{-\nabla f(x^k)}{(\nabla^2 f(x^k) + \lambda I)}$; $I = \text{Identity matrix}$

$\lambda = \text{Constant}$

λ starts with very large value : **Cauchy's Steepest descent**

Gradually reduced to low value : **Newton's Method**



$$\nabla f(X) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{bmatrix}$$

$$\nabla^2 f(x^{(t)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \frac{\partial^2 f}{\partial x_N \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}_{x^{(t)}}$$

Algorithm

Step 1 Choose a maximum number of iterations M to be performed, an initial point $x^{(0)}$, two termination parameters ϵ_1 , ϵ_2 , and set $k = 0$.

Step 2 Calculate $\nabla f(x^{(k)})$, the first derivative at the point $x^{(k)}$.

Step 3 If $\|\nabla f(x^{(k)})\| \leq \epsilon_1$, **Terminate**;

Else if $k \geq M$; **Terminate**;

Else go to Step 4.

Step 4 Perform a unidirectional search to find $\alpha^{(k)}$ using ϵ_2 such that $f(x^{(k+1)}) = f(x^{(k)} - \alpha^{(k)}\nabla f(x^{(k)}))$ is minimum. One criterion for termination is when $|\nabla f(x^{(k+1)}) \cdot \nabla f(x^{(k)})| \leq \epsilon_2$.

Step 5 Is $\frac{\|x^{(k+1)} - x^{(k)}\|}{\|x^{(k)}\|} \leq \epsilon_1$? If yes, **Terminate**;

Else set $k = k + 1$ and go to Step 2.

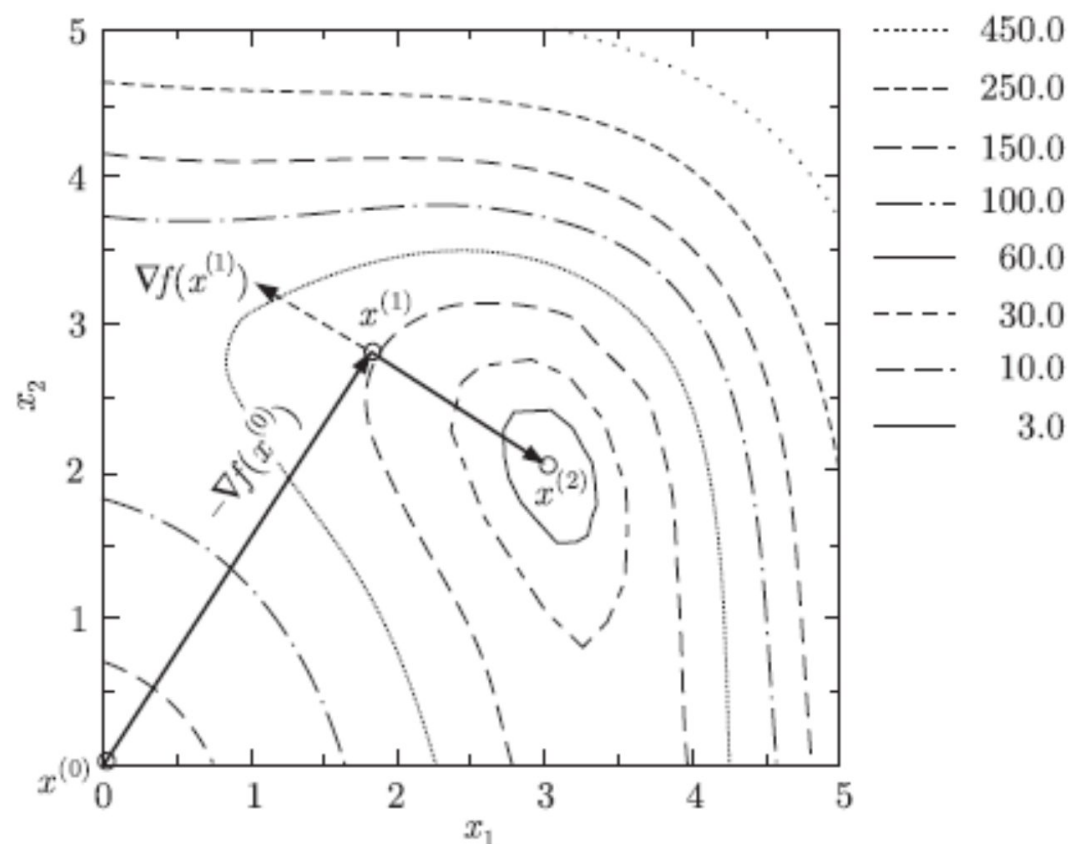
Since the direction $s^{(k)} = -\nabla f(x^{(k)})$ is a descent direction, the function value $f(x^{(k+1)})$ is always smaller than $f(x^{(k)})$ for positive values of $\alpha^{(k)}$.



$$\text{Minimize } f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2.$$

Step 1 In order to ensure proper convergence, a large value of M (≈ 100) is usually chosen. The choice of M also depends on the available time and computing resource. Let us choose $M = 100$, an initial point $x^{(0)} = (0, 0)^T$, and termination parameters $\epsilon_1 = \epsilon_2 = 10^{-3}$. We also set $k = 0$.

Step 2 The derivative at $x^{(0)}$ is first calculated using Equation (3.4) and found to be $(-14, -22)^T$, which is identical to the exact derivative at that point (Figure 3.12).



Step 3 The magnitude of the derivative vector is not small and $k = 0 < M = 100$. Thus, we do not terminate; rather we proceed to Step 4.

Step 4 At this step, we need to perform a line search from $x^{(0)}$ in the direction $-\nabla f(x^{(0)})$ such that the function value is minimum. Along that direction, any point can be expressed by fixing a value for the parameter $\alpha^{(0)}$ in the equation:

$$x = x^{(0)} - \alpha^{(0)} \nabla f(x^{(0)}) = (14\alpha^{(0)}, 22\alpha^{(0)})^T.$$

Using the golden section search (Section 2.3.3) in the interval⁵ $(0, 1)$, we obtain $\alpha^{(0)*} = 0.127$. Thus, the minimum point along the search direction is $x^{(1)} = (1.788, 2.810)^T$.

Step 5 Since, $x^{(1)}$ and $x^{(0)}$ are quite different, we do not terminate; rather we move back to Step 2. This completes one iteration of Cauchy's method. The total number of function evaluations required in this iteration is equal to 30.

Step 2 The derivative vector at this point, computed numerically, is $(-30.707, 18.803)^T$.

Step 3 This magnitude of the derivative vector is not smaller than ϵ_1 . Thus, we continue with Step 4.

Step 4 Another unidirectional search along $(30.707, -18.803)^T$ from the point $x^{(1)} = (1.788, 2.810)^T$ using the golden section search finds the new point $x^{(2)} = (3.008, 1.999)^T$ with a function value equal to 0.018.

We continue this process until one of the termination criteria is satisfied.



Marquardt's Method :

$$\nabla f(X) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{bmatrix}$$

$$H = \nabla^2 f(x^{(t)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \frac{\partial^2 f}{\partial x_N \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}_{x^{(t)}}$$

Algorithm

Step 1 Choose a starting point, $x^{(0)}$, the maximum number of iterations, M , and a termination parameter, ϵ . Set $k = 0$ and $\lambda^{(0)} = 10^4$ (a large number).

Step 2 Calculate $\nabla f(x^{(k)})$.

Step 3 If $\|\nabla f(x^{(k)})\| \leq \epsilon$ or $k \geq M$? **Terminate**;
Else go to Step 4.

Step 4 Calculate $s(x^{(k)}) = -[H^{(k)} + \lambda^{(k)}I]^{-1}\nabla f(x^{(k)})$. Set $x^{(k+1)} = x^{(k)} + s(x^{(k)})$.

Step 5 Is $f(x^{(k+1)}) < f(x^{(k)})$? If yes, go to Step 6;
Else go to Step 7.

Step 6 Set $\lambda^{(k+1)} = \frac{1}{2}\lambda^{(k)}$, $k = k + 1$, and go to Step 2.

Step 7 Set $\lambda^{(k)} = 2\lambda^{(k)}$ and go to Step 4.

A large value of the parameter λ is used initially. Thus, the Hessian matrix has little effect on the determination of the search direction (see Step 4 of the above algorithm). Initially, the search is similar to that in Cauchy's method. After a number of iterations (when hopefully the current solution has converged close to the minimum), the value of λ becomes small and the effect is more like that in Newton's method. The algorithm can be made faster by performing a unidirectional search while finding the new point in Step 4: